SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1703**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *K.Pujitha*

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Constraint-Based Problem Solving using Backtracking Search (10 Marks)

Given

Doctors: **D1, D2, D3**

Shifts:

- Morning (M)
- Afternoon (A)
- Night (N)

Constraints

1. D1 cannot work Night.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor per shift.
4. D3 cannot work Morning.
5. Every doctor must be assigned exactly one shift.

Backtracking Search

Step 1: Assign D1

Possible shifts:

- Morning ✓
- Afternoon ✓
- Night ✗ (Constraint 1)

Choose **D1 → Morning**

Current Assignment:

Doctor	Shift
D1	Morning

Constraint Check:

- D1 not Night ✓
- No duplicate

Step 2: Assign D2

Remaining shifts:

- Afternoon
- Night

Try **D2 → Afternoon**

Current Assignment

Doctor	Shift
D1	Morning
D2	Afternoon

Constraint Check:

- One doctor per shift ✓
- D2 before D3 (to be checked later)

Step 3: Assign D3

Remaining shift:

- Night

Try **D3 → Night**

Constraint Check:

- D3 not Morning ✓
- D2 before D3
  - Afternoon < Night ✓
- One doctor per shift ✓
- Every doctor assigned ✓

All constraints satisfied.

Backtracking Tree

```
Start
│
├── D1 → Morning
│       │
│       ├── D2 → Afternoon
│       │       │
│       │       └── D3 → Night ✓ Solution
│       │
│       └── D2 → Night ✗
│            (D3 only Afternoon → D2 before D3 violated)
│
└── D1 → Afternoon
        │
        ├── D2 → Morning
        │       │
        │       └── D3 → Night ✓ Another valid solution
        │
        └── D2 → Night ✗
```

Final Valid Schedule

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Final Answer:
Morning → D1
Afternoon → D2
Night → D3

2. BFS Search Algorithm (10 Marks)

5 × 5 Grid

```
S   .   .   X   .
.   X   .   X   .
.   X   .   .   .
.   .   X   X   .
.   .   .   .   G
```

Legend:

- S = Start
- G = Goal
- X = Obstacle

Movement:

- Up
- Down
- Left
- Right

Cost = 1

Heuristic:

- Manhattan Distance

Step 1

Queue:

```
[S]
```

Expand **S(0,0)**

Neighbors:

- (0,1)
- (1,0)

Queue:

```
[(0,1),(1,0)]
```

Step 2

Expand (0,1)

New Node:

```
(0,2)
```

Queue

```
[(1,0),(0,2)]
```

Step 3

Expand (1,0)

New Node

```
(2,0)
```

Queue

```
[(0,2),(2,0)]
```

Step 4

Expand (0,2)

New Node

```
(1,2)
```

Queue

```
[(2,0),(1,2)]
```

Continue Expansion

Visited Order

```
S
↓

(0,1)

↓

(1,0)

↓

(0,2)

↓

(2,0)

↓
```

(1,2)

↓

(2,2)

↓

(2,3)

↓

(2,4)

↓

(3,4)

↓

Goal (4,4)

Optimal Path

S
↓

(0,1)

↓

(0,2)

↓

(1,2)

↓

(2,2)

```
↓

(2,3)

↓

(2,4)

↓

(3,4)

↓

G
```

Path Cost

Moves = **8**

Cost per move = **1**

Total Cost = 8

Result

Optimal Path:

`S → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → G`

Total Cost = 8

3. Autonomous Rescue Robot (10 Marks)

(a) Constraint Analysis

1. Move only Up, Down, Left, Right

The robot can move only in four directions. Diagonal movement is not allowed.

2. Obstacles

Blocked cells cannot be crossed. The robot must search for an alternate path.

3. Limited Sensing

The robot can observe only adjacent rooms. It gradually builds knowledge about the environment while moving.

4. Movement Cost

Each movement costs **1**, so shorter paths are preferred.

5. Risky Zones

Entering risky areas adds **2** extra cost. The robot should avoid risky cells whenever possible.

6. Online Search

The robot updates its map continuously as it discovers new obstacles or risks and replans its route if necessary.

(b) Solution Using Uniform Cost Search (UCS) / Online Search Agent

Step 1

Start at **S**.

Step 2

Observe adjacent cells.

Step 3

Calculate total path cost.

Normal move:

```
Cost = 1
```

Risky cell:

```
Cost = 1 + 2 = 3
```

Step 4

Expand the node with the **lowest cumulative cost**.

Step 5

Update knowledge when new obstacles or risky zones are detected.

Step 6

Continue until the robot reaches **Goal (G)** with the minimum total cost.

(c) AI Concepts

Intelligent Agent

- Perceives surroundings using sensors.
- Takes actions using actuators.
- Continuously updates its knowledge.

Rationality

The robot always selects the action that minimizes total travel cost while avoiding unnecessary risks.

Environment Type

- Partially Observable (cannot see the whole building)
- Dynamic (new information appears during movement)
- Sequential (each action affects the next)
- Discrete (moves between rooms)
- Single Agent (only one rescue robot)

(d) Justification

Uniform Cost Search (UCS) combined with an Online Search Agent is suitable because:

- Finds the minimum-cost path.
- Considers both movement and risky-zone costs.
- Adapts to newly discovered obstacles.
- Works effectively in partially observable environments.
- Ensures safe and efficient navigation to rescue survivors.

Conclusion

The rescue robot uses **Uniform Cost Search with Online Search** to dynamically update its knowledge, avoid obstacles and risky areas, and reach the survivor with the minimum possible cost. This makes it the most suitable approach for disaster rescue scenarios.